Using Claude Code to build the RATIO artefact

A record of how an AI coding assistant was used to produce the Stage 1 customer facing artefact: a Next.js landing page and meal plan builder, deployed to Cloudflare Workers.

1. What was built

A working web application rather than a mockup: a landing page, a four step plan builder that prices a real order, a reservation form that writes to a database, and an analytics dashboard reporting where visitors drop out of the funnel. The nutrition model, pricing and every published figure are computed from data files, so the marketing copy cannot contradict the product.

2. How the tool was actually used

The pattern that worked was not asking for finished features. It was stating an outcome, then repeatedly asking for evidence that the outcome had been met. Most of the value came from the second half of that loop.

Prompt type	Example of what was asked	What it produced
Build	Build a landing page and plan builder for a high protein, low carb meal prep business, using a file as the database	The initial application: pages, nutrition model, pricing, storage layer
Adversarial test	Debug this / test it properly	Malformed API requests, concurrent writes, and boundary values, rather than a happy path walkthrough
Critique	The food looks fake / this is too big on mobile / the spacing is uneven	Specific measured causes rather than general polish
Evidence	Research real competitors and price them from their own sites	Named competitors with real prices, which changed the positioning
Deploy	Put this on a public URL	Static export, then a full server deployment with a real database

3. Where the AI added real value

- **Checking a rule against its whole input space.** The tray sizing logic was swept across all 36 combinations of goal, calorie target and meals per day. Six combinations were more than 10 percent off target, including a common case that was 16 percent under. A demonstration of one happy path would never have surfaced this. A fourth tray size took it to 30 of 36 within tolerance.
- **Adversarial testing on request.** Asking it to attack the reservation API produced ten malformed requests, all correctly rejected, plus a concurrency test: twelve simultaneous submissions, all twelve stored with unique codes and no corruption.
- **Primary source research.** Competitor prices and macros were read from each company's own live site rather than a comparison page, which is what made the resulting comparison defensible.

4. Where the AI was wrong, and what changed

This section matters more than the previous one. Every item below was a real error that had to be caught and corrected.

- **It wrote a confident claim it had not checked.** The landing page originally claimed superior protein density as a selling point. The figure was arithmetically correct, but no competitor had been looked at. Reading My Muscle Chef's own published macros showed 10.2g of protein per 100 kcal against our 10.6g: the advantage did not exist. The claim was removed and replaced with carbohydrate content, which the evidence does support. **The lesson is that a model will produce a fluent, arithmetically correct comparison that was never actually made.** That is harder to catch than an invented statistic, because every number in it is real.

- **It reported a friendlier number than the truth.** The builder told customers how many calories were left for them to eat, calculated from the plan's assumed coverage rather than the meals actually chosen. On a real basket it said 475 kcal remaining when the true figure was 370. Now computed from the selection.
- **It solved the wrong problem twice.** Told the 3D render was too big on mobile, it twice reduced the download size. The actual complaint was physical size: the tray overflowed the screen. The cause was that a 3D field of view is vertical, so on a narrow phone the horizontal view collapses and a wide object runs off both edges.
- **It removed a feature instead of optimising it.** Asked to optimise the mobile render, it deleted it from mobile entirely, and acted on an uncompressed file size that was three times the real transfer cost. Restored and optimised properly.
- **It broke the deployed site without noticing.** Repeated rebuilds left the running server serving a stale manifest, so every stylesheet request returned a server error and the live site rendered as unformatted HTML. Only found by inspecting the response, not by trusting that the build succeeded.

5. The constraint that was held throughout

The brief states that AI generated evidence, simulated customers and invented survey data are academic misconduct. No customer quote, survey response or market statistic in this project was generated. The quotes file ships empty and the page renders nothing rather than a plausible placeholder, and five named evidence gaps are recorded as gaps. A preflight check refuses to pass while they are open.

6. Verification practice

The single most useful habit was refusing to accept that something worked because it had been built. Claims were checked by measurement: response codes and content types for the deployed site, a database query to confirm a reservation actually persisted, decoding an encoded file to confirm its contents, and sweeping a parameter space rather than testing one case. Several of the errors in section 4 were found this way and would otherwise have shipped.

Prepared as a record of AI use for this project. Verify each claim against your own build before submitting, and edit into your own voice.